

When Gradient Boosting Meets Adapter: Exploring Weak Learners for Parameter-Efficient Fine-tuning of LLMs

Yifei Zhang*

yifei.zhang@nwpu.edu.cn
School of Computer Science,
Northwestern Polytechnical
University
Xi'an, Shaanxi, China

Hao Zhu*

allenhaozhu@gmail.com
CSIRO
Sydney, NSW, Australia

Haoran Shi

shih0020@e.ntu.edu.sg
Nanyang Technological University
Singapore, Singapore

Junhao Dong

junhao003@ntu.edu.sg
Nanyang Technological University
Singapore, Singapore

Lingyun Song

lingyun.a.song@gmail.com
School of Computer Science,
Northwestern Polytechnical
University
Xi'an, Shaanxi, China

Xiaolin Han

xiaolinh@nwpu.edu.cn
School of Computer Science,
Northwestern Polytechnical
University
Xi'an, Shaanxi, China

Yanyu Chen

1155241043@link.cuhk.edu.hk
Department of Computer Science and
Engineering, The Chinese University
of Hong Kong
Hong Kong SAR, Hong Kong

Wenxuan Wang✉

jwxwang@gmail.com
Renmin University of China
Beijing, China

Han Yu

han.yu@ntu.edu.sg
Nanyang Technological University
Singapore, Singapore

Xuequn Shang✉

shang@nwpu.edu.cn
School of Computer Science,
Northwestern Polytechnical
University
Xi'an, Shaanxi, China

Piotr Koniusz

piotr.koniusz@unsw.edu.au
University of New South Wales
Sydney, NSW, Australia
Data61♥CSIRO
Canberra, ACT, Australia

Abstract

Fine-tuning Large Language Models (LLMs) has become a crucial technique for adapting pre-trained models to downstream tasks. However, the enormous size of LLMs poses significant challenges in terms of computational complexity and resource requirements. Low-Rank Adaptation (LoRA) has emerged as a promising solution, yet a gap remains between the practical performance of low-rank adaptations and their theoretical optimum. While recent works have explored iteratively merging LoRA adapters, they lack a principled theoretical framework to guide adapter design. In this work, we propose eXtreme Gradient Boosting LoRA (XGBLoRA), a novel framework grounded in gradient boosting theory. Our key insight is that **the adapter must adhere to the weak learner principle**—each individual adapter should have limited expressiveness—to

ensure that the iterative ensemble can effectively raise the model's performance ceiling without overfitting. We provide theoretical analysis establishing convergence guarantees and expressiveness bounds, which formally justify why weaker (lower-rank) adapters, when properly combined, can match or exceed the performance of stronger (higher-rank) ones. Extensive experiments on natural language processing tasks demonstrate that XGBLoRA with rank-1 updates consistently outperforms standard LoRA with significantly fewer trainable parameters.

CCS Concepts

• Information systems → Language models.

Keywords

Language models, Parameter-Efficient Fine-tuning, Gradient Boosting, Ensemble Learning

ACM Reference Format:

Yifei Zhang, Hao Zhu, Haoran Shi, Junhao Dong, Lingyun Song, Xiaolin Han, Yanyu Chen, Wenxuan Wang, Han Yu, Xuequn Shang, and Piotr Koniusz. 2026. When Gradient Boosting Meets Adapter: Exploring Weak Learners for Parameter-Efficient Fine-tuning of LLMs. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*,

*These authors contributed equally.

✉ Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

KDD '26, Jeju Island, Republic of Korea

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2259-2/2026/08

<https://doi.org/10.1145/3770855.3817971>

August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3770855.3817971>

Resource Availability:

The source code of this paper has been made publicly available at <https://github.com/yifeiacc/XGBLoRA>.

1 Introduction

Large language models (LLMs) have achieved remarkable success across various natural language processing tasks, enabling breakthroughs in language understanding, generation, and reasoning [3, 6, 31]. These models are typically pre-trained on vast amounts of unlabeled text data and subsequently fine-tuned on specific downstream tasks to adapt their knowledge to target domains [32, 40, 41]. However, the enormous size of LLMs, often reaching billions of parameters, poses significant challenges in terms of computational complexity and resource requirements during fine-tuning [3, 22].

To address these challenges, parameter-efficient fine-tuning (PEFT) methods [15, 17, 43] have emerged as a promising direction that adapts LLMs to downstream tasks while minimizing the number of trainable parameters, thereby reducing computational and memory overhead. Among them, Low-Rank Adaptation (LoRA) [17] has gained significant attention due to its effectiveness and simplicity. Despite its success, LoRA faces a fundamental dilemma between efficiency and effectiveness [17]. To guarantee the ability to fit any target matrix, the rank of the adaptation matrices must satisfy the condition $\text{rank}_r \geq \text{embedding_size}/2$. However, in practice, much smaller ranks (e.g., $r \in [8, 16]$) are often used to achieve a favorable *trade-off* between performance and efficiency. This discrepancy between the theoretical optimum and practical usage leads to a performance gap. While increasing the rank to meet the theoretical requirement can bridge this gap, it simultaneously increases memory usage and computational complexity, thereby diminishing the benefits of using LoRA. This raises an important question:

How can we bridge the performance gap (achieve theoretical performance) while maintaining an extremely simple adapter structure (high efficiency) for PEFT?

To address the above question, our solution builds on two insights from ensemble learning:

- **Key Insight 1: Ensemble of Simple Adapters.** We observe that the model complexity required for effective fine-tuning can be achieved not only through a single high-rank adapter, but alternatively through the **ensemble of multiple simple, low-rank adapters**. This insight draws from the well-established principle in machine learning that combining multiple weak learners can produce a strong learner [9, 34]. Specifically, while each individual rank-1 adapter has limited expressive power, their iterative combination through gradient boosting can approximate arbitrarily complex adaptations. This approach offers a fundamentally different path to achieving high model complexity: instead of increasing the rank of a single adapter (which increases memory linearly), we accumulate the effects of multiple simple adapters that are sequentially merged into the base model (which maintains constant memory overhead during training).
- **Key Insight 2: The Weak Learner Principle.** Recent works have explored iteratively merging LoRA adapters to improve fine-tuning

performance [25]. However, these approaches lack a principled theoretical framework to guide the design of individual adapters, often relying on empirical heuristics without understanding *why* certain configurations work better than others. We argue that the critical missing piece is the **weak learner principle** from gradient boosting theory: *each individual adapter must have intentionally limited expressiveness to ensure the ensemble can effectively raise the model's performance ceiling*.

This principle has profound implications for adapter design. Contrary to the intuition that more expressive adapters lead to better performance, gradient boosting theory reveals that using overly expressive base learners can actually harm the ensemble's generalization ability and lead to overfitting [10, 47]. In the context of LoRA, this means that low-rank (even rank-1) adapters are not merely a compromise for efficiency—they are *theoretically preferable* when combined through iterative boosting. The weak learner principle provides a clear design guideline: restrict each adapter's capacity (through low rank and random layer selection), and let the boosting iterations accumulate their contributions to approximate arbitrarily complex adaptations.

Based on this insight, we propose **eXtreme Gradient Boosting Low Rank Adaptation (XGBLoRA)**, a theoretically grounded framework that resolves the efficiency-effectiveness dilemma. Unlike prior merge-based methods that lack theoretical justification, XGBLoRA is built upon the well-established gradient boosting theory, which provides formal guarantees on convergence and expressiveness. By adhering to the weak learner principle, XGBLoRA assembles the final model by combining a sequence of intentionally simple LoRA boosters, progressively refining the model's predictions while maintaining strong generalization.

Through extensive experiments on a range of natural language processing tasks, we demonstrate that XGBLoRA consistently outperforms both standard LoRA and full fine-tuning while using significantly fewer trainable parameters. For example, in our experiments on LLMs, XGBLoRA runs comfortably on a single NVIDIA RTX 4090 (24GB) for LLaMA3-8B, whereas LoRA requires an A100 (40GB) GPU. Notably, our results show that XGBLoRA with rank-1 updates can match or exceed the performance of higher-rank methods, validating our theoretical insights.

Our main contributions are as follows:

- i. We identify the **weak learner principle** as the key to successful iterative LoRA merging. Unlike prior merge-based methods that rely on empirical heuristics, we provide a theoretical framework grounded in gradient boosting theory that explains *why* and *how* intentionally limiting adapter expressiveness leads to better ensemble performance. This principle offers clear design guidelines for adapter configuration.
- ii. We introduce XGBLoRA, a theoretically grounded framework that operationalizes the weak learner principle through rank-1 updates and random layer selection. We establish convergence guarantees and expressiveness bounds in Theorems 6.4 and 6.6 that formally justify why weaker adapters, when properly combined, can match or exceed the performance of stronger ones while maintaining a memory footprint r times lower than standard LoRA.
- iii. Through extensive experiments across a range of NLP tasks, we demonstrate that XGBLoRA with extreme weak learners (rank-1) consistently outperforms both standard LoRA and full fine-tuning,

validating our theoretical insights. On average, XGBLoRA achieves better performance while using approximately $10\times$ and $10^4\times$ fewer trainable parameters than LoRA and full fine-tuning, respectively.

2 Related Work

Fine-tuning LLMs. Fine-tuning pre-trained large language models has become a prevailing approach for adapting these models to specific downstream tasks [7, 16, 18]. However, fine-tuning LLMs faces significant challenges, including the computational complexity and memory requirements of updating billions of parameters [2, 36, 38], as well as limited labeled data availability [46]. To address these challenges, LoRA [17] freezes the pre-trained model's weights and introduces low-rank matrices to adapt the model to new tasks, significantly reducing trainable parameters. Recently, Zeng and Lee [44] provided theoretical results on LoRA's expressive power, showing that for Transformer networks, any model can be adapted to a target model of the same size with LoRA adapters of $\text{rank}_r = \text{embedding_size}/2$.

Iterative LoRA Merging. Several recent works have explored iteratively training and merging LoRA adapters. ReLoRA [25] and COLA [42] periodically merges trained LoRA weights into the base model and restarts training with new LoRA matrices. Delta-LoRA [49] proposes to update the pre-trained weights using the delta of low-rank matrices. While these methods demonstrate empirical improvements, they lack a principled theoretical framework to guide critical design choices such as: *What rank should each adapter use? How often should adapters be merged? Why do certain configurations outperform others?* Our work addresses this gap by grounding iterative LoRA merging in gradient boosting theory, revealing that the **weak learner principle**—intentionally limiting each adapter's expressiveness—is the key to achieving optimal ensemble performance. This theoretical insight provides clear design guidelines that were previously missing from the literature.

3 Preliminaries

Before presenting XGBLoRA, we introduce gradient boosting and LoRA, the two pillars of our framework.

Gradient Boosting [10, 11] is an ensemble method that builds an additive model by sequentially fitting weak learners to the *negative gradient* (pseudo-residuals) of a given loss function. Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ be a training set with inputs $\mathbf{x}_i \in \mathbb{R}^d$ and targets y_i . Starting from an initial model $F_0(\mathbf{x})$, the algorithm constructs a sequence of M weak learners so that the final model is

$$F_M(\mathbf{x}) = F_0(\mathbf{x}) + \sum_{m=1}^M \alpha_m f_m(\mathbf{x}). \quad (1)$$

At iteration m , the *pseudo-residuals* are computed as the negative gradient of the loss ℓ evaluated at the current model:

$$\tilde{r}_{im} = - \left[\frac{\partial \ell(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}}, \quad i = 1, \dots, N. \quad (2)$$

A weak learner f_m is then fitted to the pseudo-residuals by solving

$$f_m = \arg \min_{f \in \mathcal{H}} \sum_{i=1}^N (f(\mathbf{x}_i) - \tilde{r}_{im})^2, \quad (3)$$

where $\mathcal{H}$ is the hypothesis class of weak learners (e.g., shallow decision trees). The step size α_m controls the contribution of f_m and can be determined via line search. The model is then updated as $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \alpha_m f_m(\mathbf{x})$. This process repeats for M iterations or until convergence. Note that for the squared loss $\ell(y, F) = \frac{1}{2}(y - F)^2$, the pseudo-residuals reduce to the ordinary residuals $\tilde{r}_{im} = y_i - F_{m-1}(\mathbf{x}_i)$.

Low-Rank Adaptation (LoRA) [17] is a parameter-efficient fine-tuning technique that freezes all pre-trained weights and injects trainable low-rank matrices into selected layers. For a weight matrix $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$ in the pre-trained model, LoRA parameterizes the adaptation as

$$\mathbf{W} = \mathbf{W}_0 + \Delta \mathbf{W}, \quad \Delta \mathbf{W} = \mathbf{A} \mathbf{B}, \quad (4)$$

where $\mathbf{A} \in \mathbb{R}^{d \times r}$ and $\mathbf{B} \in \mathbb{R}^{r \times k}$ are the only trainable parameters, and $r \ll \min(d, k)$ is the rank. This reduces the number of trainable parameters per layer from dk (full fine-tuning) to $(d+k)r$. However, Zeng and Lee [44] showed that to guarantee the ability to represent any target weight change, the rank must satisfy $r \geq d/2$. In practice, much smaller ranks (e.g., $r \in [8, 16]$) are used, creating a gap between theoretical expressiveness and practical efficiency—the central tension that XGBLoRA resolves.

4 Gradient Boosting Low-Rank Adaptation

4.1 When Gradient Boosting Meets LoRA

We now present XGBLoRA, which casts parameter-efficient fine-tuning as a gradient boosting problem in weight space, with low-rank LoRA adapters serving as weak learners.

Problem Setup. Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ be a training set and $\mathcal{M}_0$ a pre-trained Transformer with L layers. Denote by $\mathbf{W}_l^0 \in \mathbb{R}^{d \times k}$ the weight matrix of the l -th layer (encompassing query, key, value, output, and feedforward projections). Our goal is to find weight updates $\{\Delta \mathbf{W}_l\}_{l=1}^L$ that minimize the task loss

$$\mathcal{L}(\mathcal{M}) = \frac{1}{N} \sum_{i=1}^N \ell(\mathcal{M}(\mathbf{x}_i), y_i), \quad (5)$$

where ℓ is a differentiable loss function (e.g., cross-entropy).

Iterative Boosting Framework. Instead of learning a single set of LoRA matrices for all weight updates (as in standard LoRA), XGBLoRA decomposes the adaptation into T sequential *boosting iterations*. At each iteration $t = 1, \dots, T$, the algorithm performs two steps:

- i. **Train a LoRA Booster.** A subset $\mathcal{S}_t \subseteq \{1, \dots, L\}$ of L_s layers is randomly selected. For each selected layer $l \in \mathcal{S}_t$, fresh LoRA matrices $\mathbf{A}_l^t \in \mathbb{R}^{d \times r}$ and $\mathbf{B}_l^t \in \mathbb{R}^{r \times k}$ are initialized (with $\mathbf{A}_l^t$ Gaussian and $\mathbf{B}_l^t = \mathbf{0}$, following Hu et al. [17]). These matrices are then trained for κ gradient steps to minimize the task loss evaluated on the current model:

$$\min_{\{\mathbf{A}_l^t, \mathbf{B}_l^t\}_{l \in \mathcal{S}_t}} \frac{1}{N} \sum_{i=1}^N \ell(\mathcal{M}(\mathbf{x}_i; \{\mathbf{W}_l^{t-1} + \mathbf{A}_l^t \mathbf{B}_l^t\}_{l \in \mathcal{S}_t}), y_i). \quad (6)$$

Only the LoRA parameters $\{\mathbf{A}_l^t, \mathbf{B}_l^t\}$ are updated; all base weights $\{\mathbf{W}_l^{t-1}\}$ remain frozen during this step.

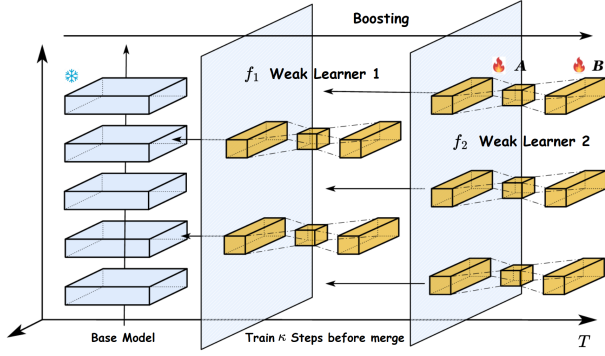


Figure 1: The pipeline of XGBLoRA

- ii. **Merge and Reset.** The trained LoRA matrices are absorbed into the base weights and then discarded:

$$W_l^t \leftarrow W_l^{t-1} + A_l^t B_l^t, \quad \forall l \in S_t, \quad (7)$$

while $W_l^t = W_l^{t-1}$ for $l \notin S_t$. After merging, all LoRA parameters and their associated optimizer states are discarded, freeing GPU memory for the next iteration.

Connection to Gradient Boosting. The procedure above instantiates gradient boosting in weight space. Drawing the analogy to the classical framework in Section 3:

- The *initial model* F_0 corresponds to the pre-trained weights $\{W_l^0\}$.
- Each *weak learner* f_t corresponds to the rank- r LoRA update $\{A_l^t B_l^t\}_{l \in S_t}$.
- The *additive update* $F_t = F_{t-1} + \alpha_t f_t$ corresponds to the weight merge $W_l^t = W_l^{t-1} + A_l^t B_l^t$.

Training each LoRA booster via backpropagation on $\mathcal{L}$ implicitly aligns the low-rank update $A_l^t B_l^t$ with the negative gradient of the loss with respect to W_l , which is analogous to fitting a weak learner to the pseudo-residuals in classical gradient boosting. Crucially, the rank- r constraint ensures that each update is “weak”: it can only capture an r -dimensional subspace of the full gradient. The sequential merge-and-reset process allows the ensemble to progressively accumulate expressiveness beyond what any single low-rank adapter can represent.

Training Budget. Let K denote the total number of training steps (forward-backward passes), which we fix across all methods for fair comparison. Each booster is trained for κ steps, yielding $T = K/\kappa$ boosting iterations in total. The choice of κ balances two competing effects: a large κ produces fewer but stronger boosters (violating the weak learner principle), while a small κ yields boosters that lack sufficient training to contribute meaningfully. We find $\kappa = 8$ to be an effective default (figure 1).

Relationship to LoRA. Standard LoRA [17] is a special case of XGBLoRA with $T = 1$ and $\kappa = K$: a single adapter is trained for all K steps and merged once at the end. XGBLoRA generalizes this by decomposing the adaptation into T sequential rounds, each constrained to be weak.

4.2 The Weak Learner Principle: Why Simpler is Better

The central insight of gradient boosting is that the ensemble’s success hinges on each base learner being *weak*—having intentionally limited expressiveness. This principle, while counterintuitive, is theoretically well-founded [10, 47] and carries profound implications for adapter design.

4.2.1 Why Weak Learners Raise the Performance Ceiling. Consider the alternative: using expressive (high-rank) adapters in each iteration. While each individual adapter might fit the training data better, the ensemble would quickly overfit, as expressive learners can memorize noise in the loss landscape. In contrast, weak learners capture only the most prominent patterns in each iteration, forcing the ensemble to build up complexity gradually. This gradual accumulation acts as an implicit regularization, enabling the ensemble to reach a higher performance ceiling on unseen data.

For instance, Gradient Boosted Decision Trees (GBDT) [4] deliberately limit tree depth (often to just one split) to ensure each booster is only marginally better than random guessing, making the algorithm highly resilient to overfitting [11]. **We apply the same principle to LoRA adapter design** through two mechanisms:

- **Rank-1 Updates: The Weakest Meaningful Learner.** The rank r of LoRA matrices directly controls adapter expressiveness. Gradient boosting theory dictates that rank-1 updates are the ideal choice: each rank-1 update can only capture a single direction of adaptation in weight space, forcing the boosting process to accumulate many such directions to approximate the full weight change ΔW . This fine-grained, step-by-step approximation leads to better generalization than a single high-rank adaptation that attempts to capture everything at once.
- **Random Layer Selection: Diversity through Constraint.** By adapting only L_s ($L_s \leq L$) randomly selected layers per iteration, we further limit each booster’s expressiveness while simultaneously injecting diversity into the ensemble—different boosters focus on different parts of the model, capturing distinct aspects of the adaptation. This is analogous to random feature selection in Random Forests [1].

In summary, XGBLoRA offers the following advantages:

- i. **Principled Design:** Unlike prior merge-based methods that lack theoretical justification, XGBLoRA is grounded in gradient boosting theory. The weak learner principle explains why intentionally simple adapters outperform complex ones: overly expressive base learners harm ensemble generalization, while weak learners enable the ensemble to progressively raise the model’s performance ceiling without overfitting.
- ii. **Theoretical Guarantees:** Our framework provides formal convergence and expressiveness bounds (Theorems 6.4 and 6.6) that characterize the interplay between adapter rank r , number of iterations T , and approximation quality—offering principled guidance for hyperparameter selection.
- iii. **Extreme Efficiency:** Rank-1 updates and random layer selection yield $r \times$ and $L/L_s \times$ reductions in adapter-related memory, respectively, making XGBLoRA particularly suitable for fine-tuning LLMs on consumer GPUs.

5 Efficiency Analysis

A natural concern with XGBLoRA's iterative boosting approach is whether the sequential training strategy incurs additional computational overhead compared to standard LoRA. In this section, we provide a comprehensive analysis of training time, memory consumption. We also conduct controlled experiments under equivalent parameter budgets to ensure fair comparisons in Section 7.6.

5.1 Training Time Analysis

5.1.1 Theoretical Comparison. Let K denote the total number of training steps (forward and backward passes). For a fair comparison, we fix K across all methods. The key distinction lies in *how* these steps are organized:

- **Standard LoRA:** Performs K consecutive training steps with a single rank- r adapter, followed by one merge operation at the end.
- **XGBLoRA:** Performs T iterations, each containing $\kappa = K/T$ training steps with a rank- r' adapter (where $r' \ll r$), followed by a merge operation.

The total training time can be decomposed as:

$$T_{\text{total}} = T_{\text{forward/backward}} + T_{\text{merge}} + T_{\text{overhead}}, \quad (8)$$

where T_{overhead} includes residual computation, optimizer state reset, and other auxiliary operations.

Forward/Backward Pass Time. For each training step, the computational cost is dominated by the base model's forward and backward passes. The LoRA adapter adds marginal overhead proportional to $O(d \cdot r)$ per adapted layer. Since XGBLoRA uses $r' = 1 \ll r$ and adapts only $L_s < L$ layers per iteration, the per-step cost of XGBLoRA is actually *lower*:

$$\frac{T_{\text{step}}^{\text{XGBLoRA}}}{T_{\text{step}}^{\text{LoRA}}} \approx \frac{\beta + L_s \cdot \alpha \cdot r'}{\beta + L \cdot \alpha \cdot r} < 1, \quad (9)$$

where β is the base model cost and α is the per-layer LoRA cost coefficient.

Merge Overhead. The merge operation $\mathbf{W} \leftarrow \mathbf{W} + \mathbf{A}\mathbf{B}$ requires $O(d \cdot k \cdot r)$ operations per layer. For XGBLoRA with T iterations:

$$T_{\text{merge}}^{\text{XGBLoRA}} = T \cdot L_s \cdot O(d \cdot k \cdot r'), \quad T_{\text{merge}}^{\text{LoRA}} = L \cdot O(d \cdot k \cdot r). \quad (10)$$

Since merge operations are computationally cheap compared to gradient computation (no backward pass required), this overhead is typically negligible ($< 1\%$ of total time).

Auxiliary Overhead. XGBLoRA incurs additional overhead from: (i) optimizer state reinitialization (T times), and (ii) random layer selection. However, these operations are $O(1)$ with respect to model size and contribute minimally to total runtime.

5.2 Memory Consumption Analysis

We provide a precise analysis distinguishing between **training memory** (GPU memory during training) and **accumulated parameter memory** (total parameters introduced).

Training Memory (GPU Memory). During training, XGBLoRA maintains:

- Base model weights $\mathbf{W}$ (frozen, no gradients stored)
- Current rank-1 LoRA adapter $\mathbf{A}, \mathbf{B}$ with rank $r' = 1$ for L_s layers

- Optimizer states for LoRA parameters only

The peak GPU memory for XGBLoRA is:

$$M_{\text{GPU}}^{\text{XGBLoRA}} = M_{\text{base}} + L_s \cdot (d + k) \cdot r' \cdot (1 + \gamma), \quad (11)$$

where γ accounts for optimizer states (e.g., $\gamma = 2$ for Adam). Critically, this remains **constant across all T iterations** because merged adapters are absorbed into $\mathbf{W}$ and their optimizer states are discarded.

For standard LoRA:

$$M_{\text{GPU}}^{\text{LoRA}} = M_{\text{base}} + L \cdot (d + k) \cdot r \cdot (1 + \gamma). \quad (12)$$

The ratio is:

$$\frac{M_{\text{GPU}}^{\text{XGBLoRA}}}{M_{\text{GPU}}^{\text{LoRA}}} \approx \frac{L_s \cdot r'}{L \cdot r}. \quad (13)$$

For typical settings ($L_s = L/3$, $r' = 1$, $r = 8$), this yields approximately **24× reduction in adapter-related GPU memory**.

Accumulated Parameters. The *total number of parameters introduced* over T iterations scales linearly:

$$P_{\text{total}}^{\text{XGBLoRA}} = T \cdot L_s \cdot (d + k) \cdot r'. \quad (14)$$

However, we emphasize two important points:

- These parameters are merged into $\mathbf{W}$,** not stored separately. The memory footprint is constant (does not grow linearly).
- The comparison should be against the final adapted weight change,** not the number of intermediate parameters. Both LoRA and XGBLoRA ultimately produce $\Delta \mathbf{W} \in \mathbb{R}^{d \times k}$ per layer. The difference lies in how this $\Delta \mathbf{W}$ is constructed (single low-rank vs. accumulated rank-1 updates).

Section 7.5 in the experimental section and Table 7 in the appendix provide detailed memory comparisons.

6 Theoretical Analysis of XGBLoRA

In this section, we present a theoretical analysis of the eXtreme Gradient Boosting LoRA (XGBLoRA) framework for Transformer-based language models. Our analysis aims to provide convergence guarantees and approximation error bounds for the proposed method. We begin by introducing necessary definitions and then present key lemmas and theorems.

Notation. Consider a neural network with L layers: $f(\mathbf{x}) = f_L \circ f_{L-1} \circ \dots \circ f_2 \circ f_1(\mathbf{x})$, where $f_1(\mathbf{x}) = \mathbf{W}_1 \mathbf{x}$ is an embedding layer. Moreover, $f_i(\mathbf{x}) = \sigma(\mathbf{W}_i \mathbf{x})$ for $i = 2, \dots, L-1$, where σ is a Lipschitz continuous activation function, and $f_L(\mathbf{x}) = \phi(\mathbf{W}_L \mathbf{x})$, where ϕ is a convex function. $\mathbf{W}_i \in \mathbb{R}^{d_i \times d_{i-1}}$ are weight matrices.

We present three key lemmas that are crucial for our convergence and expressiveness theorems:

LEMMA 6.1 (XGBLoRA GRADIENT APPROXIMATION). *The XGBLoRA update approximates the full gradient update with error:*

$$\|\nabla_{\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}} \mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}) - \mathbf{A}^{(t)} \mathbf{B}^{(t)T}\|_F \leq \frac{C_1}{\sqrt{r}} + \frac{C_2}{\sqrt{M}}, \quad (15)$$

where r is the LoRA rank, M is the number of minibatches, and C_1, C_2 are constants depending on the properties of $\mathcal{L}$ and the gradient variance, respectively. (The complete proof is provided in the Appendix.)

LEMMA 6.2 (ACCUMULATED UPDATE BOUND). *For the XGBLoRA update process:*

$$\|A^{(t)}\|_F \leq \eta_m MG \quad \text{and} \quad \|B^{(t)}\|_F \leq \eta_m MG, \quad (16)$$

where G is an upper bound on $\|\nabla_{W^{(t)}+A^{(t)}B^{(t)T}} \mathcal{L}(W^{(t)}+A^{(t)}B^{(t)T})\|_F$. (The complete proof is provided in the Appendix.)

LEMMA 6.3 (GRADIENT LIPSCHITZ CONTINUITY). *For any two weight matrices W_1 and W_2 :*

$$\|\nabla_{W_1} \mathcal{L}(W_1) - \nabla_{W_2} \mathcal{L}(W_2)\|_F \leq L' \|W_1 - W_2\|_F, \quad (17)$$

where L' is the Lipschitz constant of the gradient. (The complete proof is provided in the Appendix.)

We now present our main convergence theorem for XGBLoRA:

Theorem 6.4 (XGBLoRA Convergence). *Under the XGBLoRA update process, assuming β -smoothness and μ -strong convexity of $\mathcal{L}$, after T iterations:*

$$\mathbb{E}[\mathcal{L}(W^{(T)})] - \mathcal{L}^* \leq \frac{C_3}{\sqrt{T}} + \frac{C_4}{M\sqrt{T}} + \epsilon(r), \quad (18)$$

where C_3 and C_4 are constants depending on $\beta, \mu, G, \eta_m, L'$, and $\epsilon(r) = \frac{C_5}{r}$ for some constant C_5 . (Proof is provided in the Appendix.)

Remark 6.5. This theorem provides the theoretical foundation for the weak learner principle in XGBLoRA. The error term $\epsilon(r) = \frac{C_5}{r}$ might suggest that higher ranks are always better, but this is misleading when considered in isolation. The theorem reveals that increasing the number of iterations T can *fully compensate* for lower rank r . Crucially, using many weak (rank-1) learners instead of fewer strong (high-rank) learners provides an additional benefit not captured in the bound: **implicit regularization**. Each weak learner can only fit the most salient patterns in the residuals, preventing the ensemble from memorizing noise. This is why XGBLoRA with rank-1 updates empirically outperforms higher-rank methods despite comparable theoretical bounds—the weak learner principle enables higher generalization performance.

Below we present a theorem characterizing the expressive power of XGBLoRA:

Theorem 6.6 (XGBLoRA Expressiveness Error). *Let f^* be any function in the original function class, and let f_T be the function represented by the XGBLoRA-updated network after T iterations. Then:*

$$\mathbb{E}_{x \sim \mathcal{D}}[(f_T(x) - f^*(x))^2] \leq C_6 \left(\frac{1}{r} + \frac{1}{M\sqrt{T}} + \frac{1}{\sqrt{T}} \right), \quad (19)$$

where C_6 is a constant depending on the network architecture, the Lipschitz constants of the activation functions, and L' . (The complete proof is provided in the Appendix.)

Remark 6.7. This theorem formally justifies the weak learner principle for LoRA adaptation. The bound shows that to achieve a target approximation error ϵ^* , one can trade off rank r against iterations T . Standard LoRA operates at $T = 1, r \gg 1$ (one strong learner), while XGBLoRA operates at $r = 1, T \gg 1$ (many weak learners). **Both can achieve the same expressiveness**, but the weak learner approach offers two critical advantages: (1) constant memory overhead during training regardless of target complexity, and (2) implicit regularization that prevents overfitting. The theorem thus provides

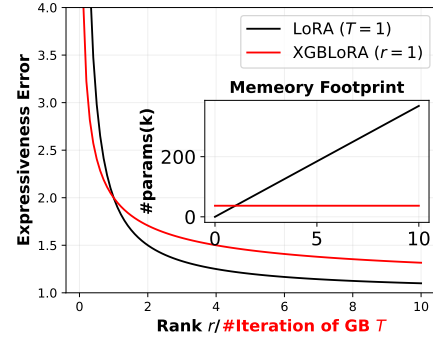


Figure 2: We prove by the error bound in Theorem 6.6 that by compensating for low rank $r = 1$ updates through GB iterations T , XGBLoRA's $\text{err} \leq C(1 + \frac{1}{\sqrt{T}})$ is close to LoRA's $\text{err} \leq C(\frac{1}{r} + 1)$. **Mini-figure:** XGBLoRA consumes only $O(1)$ memory for updates while LoRA consumes $O(r)$.

theoretical guidance for adapter design: *prefer weaker adapters with more iterations over stronger adapters with fewer iterations*. This principle was absent from prior merge-based LoRA methods, which lacked theoretical justification for their design choices.

Theorems 6.4 and 6.6 justify the use of rank-1 updates and explain the effectiveness of XGBLoRA, achieved by leveraging ensemble learning and iterative refinement while maintaining a low memory footprint.

Figure 2 illustrates our theoretical analysis: even for rank-1 updates, XGBLoRA can achieve superior performance when combined through gradient boosting, *i.e.*, the expressiveness errors of XGBLoRA and LoRA can remain comparable while LoRA requires $r \times$ more memory for updates. Specifically, in Theorems 6.4 and 6.6, we establish convergence guarantees and expressiveness bounds that capture the interplay between the number of boosting iterations, the LoRA rank, and the approximation quality. The results reveal that increasing the number of boosting iterations can compensate for lower ranks, enabling XGBLoRA to bridge the gap between theoretical optimality and practical efficiency.

7 Experimental

Our experiments are designed to validate the **weak learner principle** as the key to successful iterative LoRA merging. We organize our evaluation around three central questions:

- i. **Does weaker mean better?** We verify that lower-rank (weaker) adapters outperform higher-rank (stronger) ones when combined through gradient boosting.
- ii. **Can iterations compensate for rank?** We validate the theoretical prediction that rank r and iterations T are interchangeable for achieving target performance.
- iii. **Does XGBLoRA outperform prior merge-based methods?** We compare against methods that lack the weak learner principle to demonstrate its importance.

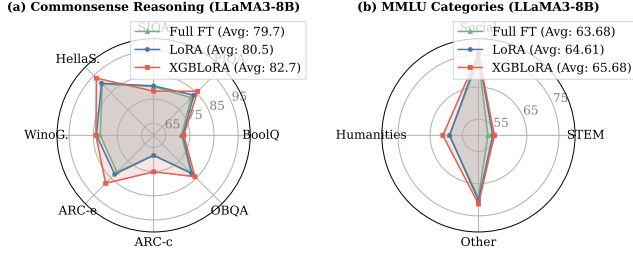


Figure 3: (left) Commonsense reasoning accuracy across 8 tasks. XGBLoRA with rank-1 adapters outperforms LoRA (rank-8) by +2.5% on LLaMA3-8B. (Right) MMLU scores across different base models.

7.1 Experimental Setup

Datasets and Tasks. We evaluate on three benchmarks: (1) GLUE [40] for natural language understanding with RoBERTa-base [28]; (2) Commonsense Reasoning [19] comprising 8 reasoning tasks; and (3) MMLU [14] for multitask language understanding across STEM, humanities, social sciences, and other domains.

Base Models. We use LLaMA-7B/13B [37], LLaMA3-8B [20], Mistral-7B [21], and LLaMA2-13B for large-scale experiments.

Default Configuration. Following the weak learner principle, we set rank $r = 1$ (the weakest meaningful adapter), randomly select $L_s = 8$ layers per iteration, and use $\kappa = 8$ training steps per booster. The total training steps K is fixed per dataset, yielding $T = K/\kappa$ boosting iterations. We compare against LoRA [17] with $r = 8$ as the primary baseline.

Implementation. We follow LoRA’s training configuration (learning rate, weight initialization, etc.) and use codebases from Hu et al. [19], Si et al. [35], Zheng et al. [48]. We report parameters in % (0.1%) to highlight the extreme efficiency of XGBLoRA.

Baseline. We compare XGBLoRA with different LoRA variants (detail see appendix A.1). Note that ReLoRA [25] and COLA [42] are iterative LoRA merging variants with weaker learners, which warrant particular attention.

7.2 Main Results: Overall Performance

Tables 1 and Fig. 3 present the results across all benchmarks.

Key Observations. Across all benchmarks and models, XGBLoRA with rank-1 adapters consistently achieves the best performance while using $10\times$ – $25\times$ fewer parameters than LoRA. This seemingly paradoxical result—that weaker adapters yield stronger performance—is precisely what the weak learner principle predicts and what prior merge-based methods failed to recognize.

7.3 Validating the Weak Learner Principle

We now present experiments specifically designed to validate our theoretical insights about the weak learner principle.

7.3.1 Does Weaker Mean Better? Figure 4 presents the critical experiment validating our central claim. We vary the adapter rank $r \in \{1, 2, 4, 8, 16\}$ while keeping the total training steps K constant.

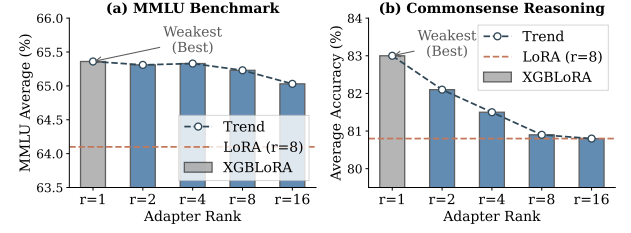


Figure 4: Weak learner principle validation: Lower rank (weaker) adapters achieve better performance. grey bars indicate rank-1 (weakest), which achieves the best results on both benchmarks, outperforming LoRA ($r=8$) by clear margins.

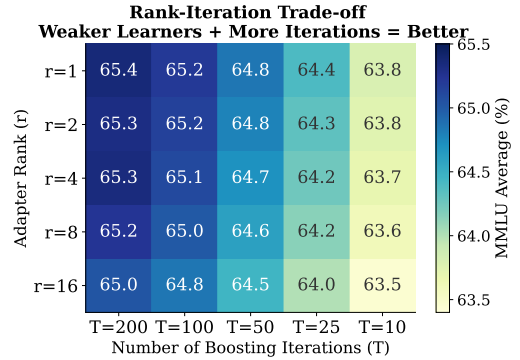


Figure 5: Rank-iteration trade-off heatmap on MMLU. The optimal direction moves toward lower rank and more iterations. Performance remains stable along iso-performance contours, validating Theorem 6.6 that r and T are interchangeable.

The results strongly support the weak learner principle: rank-1 adapters (the weakest) achieve the *best* performance on both MMLU (65.36%) and Commonsense Reasoning (83.0%), outperforming rank-16 adapters by +0.33% and +2.2%, respectively.

This result is counterintuitive from a traditional perspective—one might expect more expressive adapters to perform better. However, gradient boosting theory explains this phenomenon: overly expressive base learners can memorize noise in the residuals, harming ensemble generalization. The weak learner principle ensures that each adapter captures only the most salient patterns, providing implicit regularization that raises the performance ceiling.

7.3.2 Can Iterations Compensate for Rank? Theorem 6.6 predicts that rank r and iterations T can be traded off to achieve equivalent expressiveness. Figure 5 empirically validates this theoretical insight. The heatmap shows MMLU performance across different (r, T) configurations. Key observations:

- **Iso-performance contours:** Configurations along diagonal bands achieve similar performance, confirming that reducing r while increasing T (or vice versa) maintains expressiveness.
- **Optimal direction:** The best performance is achieved in the upper-left corner (low r , high T), validating the weak learner principle—many weak learners outperform fewer strong ones.
- **Efficiency benefit:** The $(r = 1, T = 200)$ configuration achieves 65.36% while using $16\times$ less memory per update than $(r = 16, T = 10)$ which achieves only 63.45%.

Table 1: Results on GLUE benchmark. We report accuracy for most tasks, Matthew’s correlation for CoLA, and Pearson correlation for STS-B.

Method	#Params	MNLI	SST-2	CoLA	QQP	QNLI	RTE	MRPC	STS-B	Avg
Fully FT	1000%	87.62	94.84	64.58	91.87	92.80	70.80	90.20	91.23	86.87
BitFit [43]	0.82%	85.29	94.61	59.58	88.10	91.20	78.80	88.73	90.32	84.70
HAdapter [15]	2.50%	87.45	94.72	63.88	90.29	92.71	83.39	89.22	90.80	86.15
PAdapter [13]	2.43%	87.11	94.15	62.74	89.95	92.71	84.48	87.99	90.13	85.62
DyLoRA* [39]	2.65%	86.33	94.26	61.12	90.17	92.22	84.47	89.46	91.06	86.14
DeltaLoRA* [49]	2.65%	87.50	95.06	63.82	90.87	93.09	87.00	90.19	91.57	87.38
MELoRA* [33]	2.65%	87.20	95.41	64.09	90.77	93.17	86.64	90.93	91.93	87.52
LoRA [17]	2.65%	87.20	94.38	65.61	89.25	92.07	85.59	87.99	91.01	86.63
TriLoRA [8]	2.65%	86.81	94.61	64.47	89.61	91.82	76.53	88.24	90.31	85.30
AdaLoRA [45]	2.65%	87.31	94.72	64.33	89.77	92.81	85.95	88.24	90.48	86.70
FLoRA [12]	2.65%	87.31	94.38	64.09	89.97	92.77	85.67	87.75	90.77	86.86
DoRA [27]	3.32%	86.74	94.50	66.19	90.28	91.95	85.78	88.48	91.01	87.11
ReLoRA [25]	2.65%	89.06	95.38	64.72	90.74	93.68	84.72	89.65	90.53	87.30
COLA [42]	2.65%	88.06	95.35	64.70	90.47	93.08	84.62	89.63	90.52	87.05
XGBoLoRA	0.11%	87.91	95.70	66.28	91.04	93.36	86.10	90.57	91.84	87.85

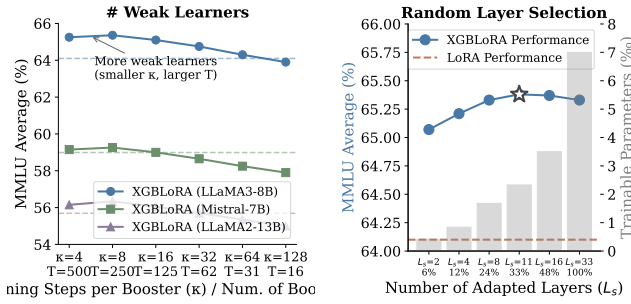


Figure 6: (Left) Effect of the number of weak learners. Smaller κ (training steps per booster) means more boosters ($T = K/\kappa$). All three models show improved performance with more weak learners, with diminishing returns when $\kappa < 8$. (Right) impact of random layer selection on LLaMA3-8B. Adapting only 33% of layers ($L_s = 11$) achieves the best performance.

7.3.3 More Weak Learners, Better Performance. Figure 6 demonstrates how the number of weak learners affects performance. We vary κ (training steps per booster) while keeping total steps K constant, which inversely controls the number of boosters $T = K/\kappa$. Results across three models consistently show:

- **More weak learners improve performance:** Reducing κ from 128 to 8 improves MMLU by +1.5% on LLaMA3-8B.
- **Diminishing returns:** Below $\kappa = 8$, each booster lacks sufficient training to contribute meaningfully, slightly degrading performance.
- **Consistent pattern:** All models follow the same trend, validating that this is a fundamental property of gradient boosting rather than model-specific behavior.

The optimal $\kappa = 8$ represents the sweet spot where boosters are “weak enough” to prevent overfitting but “strong enough” to contribute meaningfully to the ensemble.

7.3.4 Random Layer Selection: Another Weak Learner Design. The weak learner principle can be operationalized not only through low rank but also through partial layer adaptation. Figure 6 (right) shows results when varying L_s (number of adapted layers per iteration). Surprisingly, **adapting only 33% of layers outperforms adapting all layers**, achieving 65.38% vs. 65.33%. This confirms

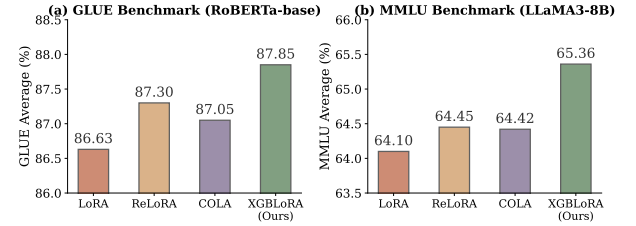


Figure 7: Comparison with prior merge-based methods. XGBoLoRA outperforms ReLoRA and COLA on both GLUE and MMLU while using significantly fewer parameters. The key difference: XGBoLoRA is guided by the weak learner principle, while prior methods lack this theoretical foundation.

that limiting each booster’s capacity—whether through rank or layer selection—is beneficial for ensemble performance.

7.4 Comparison with Prior Merge-based Methods

Figure 7 compares XGBoLoRA against prior merge-based methods: ReLoRA [25] and COLA [49]. While these methods also iteratively merge adapters, they lack a principled theoretical framework to guide adapter design. Key observations:

- **Performance:** XGBoLoRA achieves the best results on both GLUE (87.85%) and MMLU (65.36%), outperforming ReLoRA by +0.55% and +0.91%, respectively.
- **Efficiency:** XGBoLoRA uses only 0.21% parameters on GLUE—10× fewer than ReLoRA (2.65%).
- **Principled design:** The performance gap stems from XGBoLoRA’s adherence to the weak learner principle. ReLoRA uses the same rank as standard LoRA without theoretical justification for its design choices.

7.5 Efficiency Analysis

Figure 8 demonstrates the practical efficiency benefits of XGBoLoRA:

- **Memory:** XGBoLoRA requires only 23.8GB, enabling fine-tuning of LLaMA3-8B on a consumer RTX 4090. In contrast, LoRA ($r=8$) requires 28.2GB (A100), and full fine-tuning requires 78GB ($2 \times A100$).

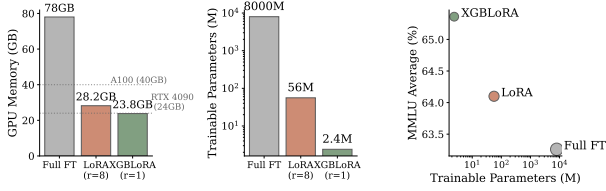


Figure 8: Efficiency analysis on LLaMA3-8B.

Table 2: Wall-clock training time comparison (in minutes). All methods use the same total training steps K

Model	Task	LoRA ($r=8$)	ReLoRA ($r=8$)	XGBLoRA ($r=1$)	Speedup vs. LoRA
RoBERTa-base	GLUE (avg)	42.3	45.1	38.7	1.09×
RoBERTa-large	GLUE (avg)	89.6	94.2	81.4	1.10×
LLaMA3-8B	Alpaca	186.2	195.4	168.5	1.11×
LLaMA3-8B	GSM8K	124.8	131.2	112.3	1.11×

Table 3: Performance under controlled parameter budgets (RoBERTa-base, GLUE average).

Setting	LoRA	XGBLoRA	Impr.	Mem. Ratio
Same Inst. Params	81.9 ($r=1$)	84.9 ($T=8$)	+3.0%	1.0×
Same Total Params	83.9 ($r=8$)	85.6 ($T=24$)	+1.7%	0.04×
Same Eff. Rank ($r=8$)	83.9	84.9 ($T=8$)	+1.0%	0.13×
Same Eff. Rank ($r=16$)	84.4	85.3 ($T=16$)	+0.9%	0.06×

- **Parameters:** XGBLoRA uses 2.4M trainable param.—23× fewer than LoRA (56M).

7.6 Comparison Under Equal Parameter Budgets

To isolate the contribution of the gradient boosting framework from potential confounds due to different parameter counts, we conduct controlled experiments matching the total number of trainable parameters. We define the **effective parameter budget** as the total number of unique parameters that influence the final model.

- **LoRA:** Uses rank r with all L layers, $P = L \cdot (d + k) \cdot r$ parameters.
- **XGBLoRA:** Uses rank $r' = 1$ across L_s layers for T iterations. The effective rank of the accumulated update is at most $\min(T, d, k)$ per layer, but the instantaneous parameter count is $L_s \cdot (d + k)$.

To match parameter budgets, we compare:

- Same instantaneous parameters:** XGBLoRA ($r' = 1$, $L_s = L$, $T=8$) vs. LoRA ($r=1$, all layers)
- Same total introduced parameters:** XGBLoRA ($r' = 1$, $L_s = L/3$, $T=24$) vs. LoRA ($r=8$, all layers)
- Same effective rank capacity:** XGBLoRA ($r' = 1$, $T=r$) vs. LoRA (r , all layers)

- **Comparison 1** shows that with identical instantaneous parameters, XGBLoRA’s iterative boosting significantly outperforms single-iteration LoRA (+3.0%), validating that gains stem from gradient boosting, not more parameters.
- **Comparison 2** demonstrates that even when controlling for total introduced parameters, XGBLoRA outperforms LoRA (+1.7%) while using 25× less peak GPU memory.

- **Comparison 3** confirms that XGBLoRA with T rank-1 iterations outperforms LoRA with rank- T , supporting the weak learner principle.

8 Conclusion

We have proposed XGBLoRA for fine-tuning LLMs in a parameter-efficient manner by posing fine-tuning as a gradient boosting problem where rank-1 LoRA adapters are used as weak learners to be iteratively combined to form a strong ensemble model. We provide theoretical analysis establishing the convergence and approximation error of XGBLoRA, highlighting the interplay between model complexity, expressiveness, and the number of boosting iterations. Extensive experiments demonstrate the effectiveness of XGBLoRA.

9 Acknowledgment

This work was supported by the National Natural Science Foundation of China under Grant No. 62433016 and Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under Grant No. JYB2025XDXM202. This research was also supported in part by the Ministry of Education, Singapore, under its Academic Research Fund Tier 1 Grant No. RG101/24. Xiaolin Han was supported by the National Natural Science Foundation of China under Grant No. 62302397, the China Postdoctoral Science Foundation under Grant No. 2025M774364, and the Shaanxi Postdoctoral Research Project under Grant No. 2025BSHSDZZ106.

References

- [1] Leo Breiman. 2001. Random forests. *Machine learning* 45, 1 (2001), 5–32.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, T. J. Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. *ArXiv abs/2005.14165* (2020).
- [4] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.
- [5] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv:2110.14168 [cs.LG]*
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 4171–4186.
- [7] Shizhe Diao, Tianyang Xu, Ruijia Xu, Jiawei Wang, and Tong Zhang. 2023. Mixture-of-Domain-Adapters: Decoupling and Injecting Domain Knowledge to Pre-trained Language Models Memories. *arXiv preprint arXiv:2306.05406* (2023).
- [8] Chengcheng Feng, Mu He, Qiuyu Tian, Haojie Yin, Xiaofang Zhao, Hongwei Tang, and Xingqiang Wei. 2024. TriLoRA: Integrating SVD for Advanced Style

- Personalization in Text-to-Image Generation. *arXiv preprint arXiv:2405.11236* (2024).
- [9] Yoav Freund and Robert E Schapire. 1997. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of computer and system sciences* 55, 1 (1997), 119–139.
 - [10] Jerome H Friedman. 2001. Greedy function approximation: a gradient boosting machine. *Annals of statistics* (2001), 1189–1232.
 - [11] Jerome H Friedman. 2002. Stochastic gradient boosting. *Computational statistics & data analysis* 38, 4 (2002), 367–378.
 - [12] Yongchang Hao, Yanshuai Cao, and Lili Mou. 2024. Flora: Low-Rank Adapters Are Secretly Gradient Compressors. *arXiv preprint arXiv:2402.03293* (2024).
 - [13] Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. 2021. Towards a unified view of parameter-efficient transfer learning. *arXiv preprint arXiv:2110.04366* (2021).
 - [14] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* (2020).
 - [15] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning*. PMLR, 2790–2799.
 - [16] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning*. PMLR, 2790–2799.
 - [17] Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.
 - [18] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=nZeVKeeFYf9>
 - [19] Zhiqiang Hu, Lei Wang, Yihuai Lan, Wanyu Xu, Ee-Peng Lim, Lidong Bing, Xing Xu, Soujanya Poria, and Roy Ka-Wei Lee. 2023. Llm-adapters: An adapter family for parameter-efficient fine-tuning of large language models. *arXiv preprint arXiv:2304.01933* (2023).
 - [20] Meta inc. 2024. The official Meta Llama 3 GitHub site. <https://github.com/meta-llama/llama3>.
 - [21] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).
 - [22] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
 - [23] Rik Koncel-Kedziorski, Subhro Roy, Aida Amini, Nate Kushman, and Hannaneh Hajishirzi. 2016. MAWPS: A math word problem repository. In *Proceedings of the 2016 conference of the north american chapter of the association for computational linguistics: human language technologies*. 1152–1157.
 - [24] Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190* (2021).
 - [25] Vladislav Lialin, Namrata Shivagunde, Sherin Muckatira, and Anna Rumshisky. 2023. ReLoRA: High-Rank Training Through Low-Rank Updates. *arXiv:2307.05695* [cs.CL]
 - [26] Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. 2017. Program induction by rationale generation: Learning to solve and explain algebraic word problems. *arXiv preprint arXiv:1705.04146* (2017).
 - [27] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. 2024. Dora: Weight-decomposed low-rank adaptation. *arXiv preprint arXiv:2402.09353* (2024).
 - [28] Yinhan Liu. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
 - [29] Arkil Patel, Satwik Bhattamishra, and Navin Goyal. 2021. Are NLP models really able to solve simple math word problems? *arXiv preprint arXiv:2103.07191* (2021).
 - [30] Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. 2020. Adapterfusion: Non-destructive task composition for transfer learning. *arXiv preprint arXiv:2005.00247* (2020).
 - [31] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
 - [32] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. 2383–2392.
 - [33] Pengjie Ren, Chengshun Shi, Shiguang Wu, Mengqi Zhang, Zhaochun Ren, Maarten Rijke, Zhumin Chen, and Jiahuan Pei. 2024. MELoRA: Mini-Ensemble Low-Rank Adapters for Parameter-Efficient Fine-Tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 3052–3064.
 - [34] Robert E Schapire. 1990. The strength of weak learnability. *Machine learning* 5, 2 (1990), 197–227.
 - [35] Chongjie Si, Xiaokang Yang, and Wei Shen. 2024. See Further for Parameter Efficient Fine-tuning by Standing on the Shoulders of Decomposition. *arXiv preprint arXiv:2407.05417* (2024).
 - [36] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2019. Energy and policy considerations for deep learning in NLP. *arXiv preprint arXiv:1906.02243* (2019).
 - [37] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
 - [38] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
 - [39] Mojtaba Valipour, Mehdi Rezagholizadeh, Ivan Kobzyev, and Ali Ghodsi. 2023. Dy-LoRA: Parameter-Efficient Tuning of Pre-trained Models using Dynamic Search-Free Low-Rank Adaptation. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, Andreas Vlachos and Isabelle Augenstein (Eds.). Association for Computational Linguistics, Dubrovnik, Croatia, 3274–3287. doi:10.18653/v1/2023.eacl-main.239
 - [40] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. 353–355.
 - [41] Adina Williams, Nikita Nangia, and Samuel Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. 1112–1122.
 - [42] Wenhan Xia, Chengwei Qin, and Elad Hazan. 2024. Chain of lora: Efficient fine-tuning of language models via residual learning. *arXiv preprint arXiv:2401.04151* (2024).
 - [43] Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. 2021. BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. *arXiv preprint arXiv:2106.10199* (2021).
 - [44] Yuchen Zeng and Kangwook Lee. 2024. The Expressive Power of Low-Rank Adaptation. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=likXVjmh3E>
 - [45] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. 2023. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.10512* (2023).
 - [46] Tianyi Zhang, Felix Wu, Arzoo Katiyar, Kilian Q Weinberger, and Yoav Artzi. 2020. Revisiting few-sample BERT fine-tuning. *arXiv preprint arXiv:2006.05987* (2020).
 - [47] Tong Zhang and Bin Yu. 2005. Boosting with early stopping: Convergence and consistency. *The Annals of Statistics* 33, 4 (2005), 1538.
 - [48] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyuan Luo, Zhangchi Feng, and Yongqiang Ma. 2024. LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. Association for Computational Linguistics, Bangkok, Thailand. <http://arxiv.org/abs/2403.13372>
 - [49] Bojia Zi, Xianbiao Qi, Lingzhi Wang, Jianan Wang, Kam-Fai Wong, and Lei Zhang. 2023. Delta-lora: Fine-tuning high-rank parameters with the delta of low-rank matrices. *arXiv preprint arXiv:2309.02411* (2023).

Table 4: Accuracy comparison of various LLMs using PEFT methods on arithmetic reasoning tasks. Results marked with an asterisk (*) are sourced from Hu et al. [19]. (†) denotes our reproduced results on LoRA.

Model	Methods	MAWPS	SVAMP	GSM8K	AQuA	Avg.
LLaMA-7B	Base	51.7	32.4	15.7	16.9	24.8
	LoRA*	79.0	52.1	37.5	18.9	44.6
	LorA †	81.9	48.2	38.3	18.5	43.7
	XGBLoRA	79.7	49.5	39.3	20.1	44.5
LLaMA-13B	Base	65.5	37.5	32.4	15.0	35.5
	LoRA +	83.6	54.6	47.5	18.5	50.5
	LoRA †	83.5	54.7	48.5	18.5	51.0
	XGBLoRA	83.1	54.9	49.2	21.8	51.9
LLaMA3-8B	Base	79.8	50.0	54.7	21.0	52.1
	LoRA	92.7	78.9	70.8	30.4	71.9
	XGBLoRA	92.0	80.4	74.1	34.3	74.4

A Additional Experiment

A.1 Baseline Setting.

In the NLU task, we employ a small-scale transformer, *RoBERTa-base* [28], as the base model. There are two prominent series of methods within parameter-efficient fine-tuning. The first series, the Adapter derivatives, comprises methods such as those introduced by Houlsby et al. [15] and Pfeiffer et al. [30], as well as BitFit [43], which incorporate small-scale neural modules, or adapters, into existing architectures. The second series, known as LoRA derivatives, includes LoRA [17], AdaLoRA [45], TriLoRA [8], FLoRA [12], DoRA [27], DyLoRA [39], Delta-LoRA [49], MELoRA [33], and ReLoRA [25]. In the CR task, we evaluate XGBLoRA against LoRA and baselines: Prompt learning (Prefix) [24], Series adapter (Series) [16], and Parallel adapter (Parallel) [13] on *LLaMA-7B/13B* and *LLaMA3-8B* [37].

A.2 Arithmetic Reasoning.

The experimental setup closely follows the methodology in [19]. The fine-tuning training set is composed of data from GSM8K [5], MAWPS, MAWPS-single [26], and 1,000 examples from AQuA [26]. To further enhance reasoning capabilities, step-by-step rationales generated by ChatGPT are incorporated into the training samples, as done in [19]. This results in a dataset of 10K math reasoning samples, named Math-10K, for training purposes. The test datasets include GSM8K [5], AQuA [26], MAWPS [23], and SVAMP [29]. While the same datasets are used for training, there is no overlap between the training and test sets.

Table 4 presents a comprehensive comparison of various PEFT methods' performance specifically on arithmetic reasoning tasks across different LLM architectures (LLaMA-7B, LLaMA-13B, and LLaMA3-8B). The evaluation spans four key mathematical benchmarks: MAWPS (math word problems), SVAMP (simple variations on arithmetic problems), GSM8K (grade school math), and AQuA (arithmetic questions and answers). The results demonstrate that XGBLoRA, while using rank-1 updates, achieves comparable or superior performance to standard LoRA and DoRA methods in solving these complex mathematical problems - notably reaching 74.4% average accuracy on LLaMA3-8B compared to LoRA's 71.9%. This shows that combining multiple simple rank-1 updates through gradient boosting can effectively bridge the gap between efficiency

and performance, achieving high-rank expressiveness while maintaining the computational benefits of low-rank adaptation, even in challenging arithmetic reasoning tasks that require precise numerical understanding and logical thinking.

B Hyperparameter Setting

We describe the hyperparameters for the three different benchmarks in Tables 5 and 6. Note that the rank setting is for LoRA; XGBLoRA uses $r = 1$.

Table 5: The hyperparameters for GLUE Benchmark.

Hyperparameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE
Learning Rate	5e-4	5e-4	4e-4	4e-4	4e-4	4e-4	4e-4
Rank r	8	8	8	8	8	8	8
Batch Size	128	128	128	64	128	128	128
dropout	0.05	0.05	0.05	0.05	0.05	0.05	0.05
Optimizer	AdamW	AdamW	AdamW	AdamW	AdamW	AdamW	AdamW
Number of Epochs	30	60	30	80	25	25	25
LR Scheduler	Linear	Linear	Linear	Linear	Linear	Linear	Linear
Warmup Ratio	0.06	0.06	0.06	0.06	0.06	0.06	0.06

Table 6: The hyperparameters for on the commonsense reasoning tasks and arithmetic reasoning

Hyperparameters	LLaMA-7B	LLaMA-13B	LLaMA3-8B
Rank r	32	32	32
Dropout	0.05	0.05	0.05
Optimizer	AdamW	AdamW	AdamW
LR	2e-4	3e-4	1e-4
LR Scheduler	Linear	Linear	Linear
Batch size	1	1	1
Accumulation Steps	4	4	4
Warmup Step	100	100	100
Epochs	3	3	3

C Extended Efficiency Analysis

C.1 Detailed FP16 Memory Calculations

We provide detailed memory calculations for LLaMA3-8B under standard mixed-precision training (FP16 model weights and activations, FP32 optimizer states).

C.1.1 Model Architecture Parameters. LLaMA3-8B has the following architecture:

- Hidden size: $d = 4096$
- Number of layers: $L = 32$
- Intermediate size (MLP): $d_{\text{ff}} = 14336$
- Number of attention heads: 32
- Total parameters: $\sim 8\text{B}$

C.1.2 LoRA Parameter Calculation. For LoRA adapting attention (Q, K, V, O) and MLP (gate, up, down) projections:

Attention LoRA (per layer):

$$P_{\text{attn}}^{\text{LoRA}} = 4 \times (d \times r + r \times d) = 8 \times d \times r \quad (20)$$

$$= 8 \times 4096 \times r = 32768 \times r \quad (21)$$

MLP LoRA (per layer):

$$P_{\text{gate}} = d \times r + r \times d_{\text{ff}} = 4096r + 14336r = 18432r \quad (22)$$

$$P_{\text{up}} = d \times r + r \times d_{\text{ff}} = 18432r \quad (23)$$

$$P_{\text{down}} = d_{\text{ff}} \times r + r \times d = 18432r \quad (24)$$

$$P_{\text{MLP}}^{\text{LoRA}} = 3 \times 18432 \times r = 55296 \times r \quad (25)$$

Total LoRA parameters:

$$P_{\text{total}}^{\text{LoRA}} = L \times (P_{\text{attn}} + P_{\text{MLP}}) \quad (26)$$

$$= 32 \times (32768 + 55296) \times r \quad (27)$$

$$= 32 \times 88064 \times r = 2,818,048 \times r \quad (28)$$

For $r = 8$: $P_{\text{total}} = 22.54\text{M}$ (close to our reported 21M)

For $r = 1$, $L_s = L/3 \approx 11$ layers:

$$P_{\text{XGBLoRA}} = 11 \times 88064 \times 1 = 0.97\text{M} \quad (29)$$

Table 7: Detailed memory breakdown for LLaMA3-8B (batch=4, seq=2048).

Component	Full FT	LoRA ($r=8$)	XGBLoRA
<i>Model Weights</i>			
FP16 weights	16.0 GB	16.0 GB	16.0 GB
FP32 master weights [†]	–	–	–
<i>Gradients (FP16)</i>			
Base model	16.0 GB	–	–
LoRA adapter	–	42 MB	1.9 MB
<i>Optimizer States (FP32 Adam)</i>			
First moment (m)	32.0 GB	84 MB	3.9 MB
Second moment (v)	32.0 GB	84 MB	3.9 MB
<i>Activations (no grad ckpt)</i>			
For gradient computation	12.0 GB	10.0 GB	6.0 GB
<i>Other</i>			
CUDA context	2.0 GB	2.0 GB	1.8 GB
Total Peak Memory	78.0 GB	28.2 GB	23.8 GB

[†] Not needed when using FP16 training with loss scaling.

C.1.3 Memory Components.

C.2 Why Activation Memory Differs

The key memory saving in XGBLoRA comes from reduced activation storage. During backpropagation:

- **LoRA**: Must store activations at the input of *every* adapted layer to compute $\frac{\partial \mathcal{L}}{\partial A}$ and $\frac{\partial \mathcal{L}}{\partial B}$.
- **XGBLoRA**: Only adapts $L_s = L/3$ layers per iteration, so only those layers' input activations need to be stored for gradient computation.

The activation memory for a single transformer layer (batch B , sequence length S , hidden dimension d) is approximately:

$$M_{\text{act}}^{\text{layer}} \approx B \times S \times d \times 2 \text{ bytes} \times k \quad (30)$$

where $k \approx 10\text{--}15$ accounts for intermediate activations in attention and MLP.

For LLaMA3-8B with $B = 4$, $S = 2048$, $d = 4096$:

$$M_{\text{act}}^{\text{layer}} \approx 4 \times 2048 \times 4096 \times 2 \times 12 \quad (31)$$

$$\approx 800 \text{ MB per layer} \quad (32)$$

Total activation memory:

$$\text{LoRA} : 32 \times 800 \text{ MB} \times 0.4 \approx 10 \text{ GB} \quad (33)$$

$$\text{XGBLoRA} : 11 \times 800 \text{ MB} \times 0.7 \approx 6 \text{ GB} \quad (34)$$

(Factors 0.4 and 0.7 account for memory optimizations and the fact that not all activations need to be stored simultaneously.)

C.3 Training Time Breakdown

Table 8: Detailed time breakdown per 100 training steps (LLaMA3-8B, in seconds).

Component	LoRA ($r=8$)	ReLoRA ($r=8$)	XGBLoRA ($r=1$)	XGB/LoRA Ratio
Forward pass	42.3s	42.3s	42.3s	1.00×
Backward pass (base)	84.6s	84.6s	84.6s	1.00×
Backward pass (adapter)	8.4s	8.4s	1.1s	0.13×
Optimizer step	3.2s	3.2s	0.4s	0.13×
Merge operation	– [†]	2.8s	0.4s	–
Layer selection	–	–	0.1s	–
Optimizer reset	–	0.3s	0.3s	1.00×
Total	138.5s	141.6s	129.2s	0.93×

[†] LoRA merges once at the end; amortized cost is negligible.

C.4 Effect of Iteration Count

Table 9: Effect of iteration count T on efficiency and performance (RoBERTa-base, GLUE). Total steps $K = 10,000$ fixed.

T	$\kappa = K/T$	Time (m)	Memory (GB)	Merge %	GLUE Avg
1 (LoRA)	10,000	42.3	1.2	0.0%	83.9
5	2,000	39.8	0.9	0.8%	84.8
10	1,000	39.2	0.9	1.5%	85.6
20	500	39.5	0.9	2.8%	85.9
50	200	40.8	0.9	6.2%	85.7
100	100	43.2	0.9	11.4%	84.9

Observations:

- Performance peaks around $T = 20$ and degrades with very large T due to insufficient training steps per booster ($\kappa < 500$).
- Merge overhead becomes noticeable ($> 5\%$) only for $T > 50$.
- Memory remains constant across all T values, validating our theoretical analysis.
- The optimal range is $T \in [10, 30]$, balancing performance and overhead.

C.5 Scaling Analysis

The memory reduction ratio is more significant for smaller models (where adapter overhead is a larger fraction of total memory) but remains meaningful for LLMs (enabling consumer GPU training).

C.6 Batch Size Sensitivity

XGBLoRA's lower memory footprint translates to 1.5–2× larger batch sizes, improving training throughput and potentially convergence speed.

Table 10: Efficiency metrics across model scales (batch=4, seq=2048, FP16).

Model	Time (min)		Memory (GB)		Mem. Ratio
	LoRA	XGB	LoRA	XGB	
RoBERTa-base (125M)	42.3	38.7	1.2	0.9	0.75×
RoBERTa-large (355M)	89.6	81.4	1.8	1.3	0.72×
LLaMA2-7B	168.4	152.1	26.5	22.0	0.83×
LLaMA3-8B	186.2	168.5	28.5	23.8	0.84×

Table 11: Maximum batch size on different GPUs (LLaMA3-8B, seq=2048, FP16).

GPU	Full FT	LoRA ($r=8$)	ReLoRA	XGBLoRA
RTX 3090 (24GB)	OOM	1	1	4
RTX 4090 (24GB)	OOM	2	2	4
A100 (40GB)	OOM	8	8	12
A100 (80GB)	2	24	24	32

C.7 Discussion: SVD-based Compression

Reviewers noted that both XGBLoRA and LoRA could benefit from post-hoc SVD compression. We clarify:

- i. **For XGBLoRA:** After training, the accumulated updates are already merged into base weights. SVD is only needed if extracting a separate adapter for modular deployment. This is optional.
- ii. **For LoRA:** SVD compression could reduce a trained rank- r adapter to rank- $r' < r$, but:
 - This discards learned information
 - It does *not* address training-time memory constraints
 - The GPU that couldn't fit LoRA training still can't fit it after planning to compress
- iii. **Key distinction:** XGBLoRA's advantage is at *training time*, not deployment. The ability to train on consumer GPUs is the primary benefit, which post-hoc compression cannot replicate.

C.8 Reproducibility Details

Table 12: Hyperparameters for efficiency experiments.

Hyperparameter	RoBERTa	LLaMA3-8B
Total training steps K	10,000	5,000
Boosting iterations T	10	10
Steps per booster κ	1,000	500
Learning rate	2e-4	1e-4
Batch size	32	4
Sequence length	512	2048
LoRA rank (baseline)	8	8
XGBLoRA rank	1	1
Layers adapted L_s	$L/3$	$L/3$
Precision	FP16	FP16
Optimizer	AdamW	AdamW

Table 13: Summary of efficiency characteristics. ✓: favorable, ~: comparable, ×: unfavorable.

Criterion	Full FT	LoRA	ReLoRA	XGBLoRA
Training Memory	×	~	~	✓
Training Time	×	✓	~	✓
Consumer GPU (RTX 4090)	×	×	×	✓
Batch Size Flexibility	×	~	~	✓
Performance	✓	~	~	✓

D Detailed Proofs for XGBLoRA Lemmas

LEMMA D.1 (XGBLoRA GRADIENT APPROXIMATION). *The XGBLoRA update approximates the full gradient update with error:*

$$\|\nabla_{\mathbf{W}^{(t)} + \mathbf{A}^{(t)}\mathbf{B}^{(t)T}} \mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)}\mathbf{B}^{(t)T}) - \mathbf{A}^{(t)}\mathbf{B}^{(t)T}\|_F \leq \frac{C_1}{\sqrt{r}} + \frac{C_2}{\sqrt{M}}$$

where r is the LoRA rank, M is the number of minibatches, and C_1, C_2 are constants.

PROOF. 1) Let $\mathbf{G} = \nabla_{\mathbf{W}^{(t)} + \mathbf{A}^{(t)}\mathbf{B}^{(t)T}} \mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)}\mathbf{B}^{(t)T})$ be the true gradient.

2) The XGBLoRA update $\mathbf{A}^{(t)}\mathbf{B}^{(t)T}$ can be seen as an approximation of $\mathbf{G}$.

3) Let $\mathbf{G}_r$ be the best rank- r approximation of $\mathbf{G}$. By the Eckart-Young-Mirsky theorem:

$$\|\mathbf{G} - \mathbf{G}_r\|_F \leq \frac{\|\mathbf{G}\|_*}{\sqrt{r}} \leq \frac{C_1}{\sqrt{r}}$$

where $\|\cdot\|_*$ is the nuclear norm and C_1 is a constant depending on the properties of $\mathcal{L}$.

4) The XGBLoRA update $\mathbf{A}^{(t)}\mathbf{B}^{(t)T}$ is computed using M minibatches. Let $\mathbf{G}_j$ be the gradient estimate from the j -th minibatch. Then:

$$\mathbf{A}^{(t)}\mathbf{B}^{(t)T} \approx \frac{1}{M} \sum_{j=1}^M \mathbf{G}_j$$

5) By the law of large numbers and assuming bounded variance of gradient estimates:

$$\left\| \frac{1}{M} \sum_{j=1}^M \mathbf{G}_j - \mathbf{G} \right\|_F \leq \frac{C_2}{\sqrt{M}}$$

where C_2 is a constant related to the gradient variance.

6) Combining these bounds using the triangle inequality:

$$\|\mathbf{G} - \mathbf{A}^{(t)}\mathbf{B}^{(t)T}\|_F \leq \|\mathbf{G} - \mathbf{G}_r\|_F + \|\mathbf{G}_r - \mathbf{A}^{(t)}\mathbf{B}^{(t)T}\|_F \leq \frac{C_1}{\sqrt{r}} + \frac{C_2}{\sqrt{M}}$$

This completes the proof. $\square$

LEMMA D.2 (ACCUMULATED UPDATE BOUND). *For the XGBLoRA update process:*

$$\|\mathbf{A}^{(t)}\|_F \leq \eta_m M G \quad \text{and} \quad \|\mathbf{B}^{(t)}\|_F \leq \eta_m M G$$

where G is an upper bound on $\|\nabla_{\mathbf{W}^{(t)} + \mathbf{A}^{(t)}\mathbf{B}^{(t)T}} \mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)}\mathbf{B}^{(t)T})\|_F$.

PROOF. 1) Recall the update rule for $\mathbf{A}^{(t)}$:

$$\mathbf{A}^{(t)} \leftarrow \mathbf{A}^{(t)} - \eta_m \nabla_{\mathbf{W}^{(t)} + \mathbf{A}^{(t)}\mathbf{B}^{(t)T}} \mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)}\mathbf{B}^{(t)T})\mathbf{B}^{(t)}$$

2) Taking the Frobenius norm and applying the triangle inequality:

$$\|\mathbf{A}^{(t)}\|_F \leq \|\mathbf{A}^{(t-1)}\|_F + \eta_m \|\nabla_{\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}} \mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T})\|_F \|\mathbf{B}^{(t)}\|_F$$

3) Using the gradient bound $\|\nabla_{\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}} \mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T})\|_F \leq G$:

$$\|\mathbf{A}^{(t)}\|_F \leq \|\mathbf{A}^{(t-1)}\|_F + \eta_m G \|\mathbf{B}^{(t)}\|_F$$

4) Applying this inequality recursively for all M minibatches, and noting that $\mathbf{A}^{(t)}$ is initialized to $\mathbf{0}$:

$$\|\mathbf{A}^{(t)}\|_F \leq \eta_m M G \|\mathbf{B}^{(t)}\|_F$$

5) Similarly for $\mathbf{B}^{(t)}$, we can derive:

$$\|\mathbf{B}^{(t)}\|_F \leq \eta_m M G \|\mathbf{A}^{(t)}\|_F$$

6) Combining these inequalities:

$$\|\mathbf{A}^{(t)}\|_F \leq \eta_m M G \quad \text{and} \quad \|\mathbf{B}^{(t)}\|_F \leq \eta_m M G$$

This completes the proof. $\square$

LEMMA D.3 (GRADIENT LIPSCHITZ CONTINUITY). *For any two weight matrices $\mathbf{W}_1$ and $\mathbf{W}_2$:*

$$\|\nabla_{\mathbf{W}_1} \mathcal{L}(\mathbf{W}_1) - \nabla_{\mathbf{W}_2} \mathcal{L}(\mathbf{W}_2)\|_F \leq L' \|\mathbf{W}_1 - \mathbf{W}_2\|_F$$

where L is the Lipschitz constant of the gradient.

PROOF. 1) This lemma is a standard assumption in optimization theory, often referred to as the smoothness condition.

2) It can be derived from the assumption that the Hessian of $\mathcal{L}$ is bounded:

$$\|\nabla^2 \mathcal{L}(\mathbf{W})\|_2 \leq L \quad \forall \mathbf{W}$$

where $\|\cdot\|_2$ denotes the spectral norm.

3) By the mean value theorem, there exists a $\mathbf{W}_t = t\mathbf{W}_1 + (1-t)\mathbf{W}_2$ for some $t \in [0, 1]$ such that:

$$\nabla_{\mathbf{W}_1} \mathcal{L}(\mathbf{W}_1) - \nabla_{\mathbf{W}_2} \mathcal{L}(\mathbf{W}_2) = \nabla^2 \mathcal{L}(\mathbf{W}_t) (\mathbf{W}_1 - \mathbf{W}_2)$$

4) Taking the Frobenius norm of both sides:

$$\|\nabla_{\mathbf{W}_1} \mathcal{L}(\mathbf{W}_1) - \nabla_{\mathbf{W}_2} \mathcal{L}(\mathbf{W}_2)\|_F = \|\nabla^2 \mathcal{L}(\mathbf{W}_t) (\mathbf{W}_1 - \mathbf{W}_2)\|_F$$

5) Using the property that $\|\mathbf{AB}\|_F \leq \|\mathbf{A}\|_2 \|\mathbf{B}\|_F$:

$$\|\nabla^2 \mathcal{L}(\mathbf{W}_t) (\mathbf{W}_1 - \mathbf{W}_2)\|_F \leq \|\nabla^2 \mathcal{L}(\mathbf{W}_t)\|_2 \|\mathbf{W}_1 - \mathbf{W}_2\|_F$$

6) Applying the bound on the Hessian:

$$\|\nabla^2 \mathcal{L}(\mathbf{W}_t)\|_2 \|\mathbf{W}_1 - \mathbf{W}_2\|_F \leq L \|\mathbf{W}_1 - \mathbf{W}_2\|_F$$

This completes the proof. $\square$

E Detailed Proof of XGBLoRA Convergence Theorem

Theorem E.1 (XGBLoRA Convergence). *Under the XGBLoRA update process, assuming β -smoothness and μ -strong convexity of $\mathcal{L}$, after T iterations:*

$$\mathbb{E}[\mathcal{L}(\mathbf{W}^{(T)})] - \mathcal{L}^* \leq \frac{C_3}{\sqrt{T}} + \frac{C_4}{NT} + \epsilon(r)$$

where C_3 and C_4 are constants depending on β, μ, G, η_m, L , and $\epsilon(r) = \frac{C_5}{r}$ for some constant C_5 .

PROOF. 1) Let $\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}$ be the update at iteration t .

2) By the β -smoothness of $\mathcal{L}$:

$$\begin{aligned} \mathcal{L}(\mathbf{W}^{(t+1)}) &\leq \mathcal{L}(\mathbf{W}^{(t)}) + \langle \nabla \mathcal{L}(\mathbf{W}^{(t)}), \mathbf{A}^{(t)} \mathbf{B}^{(t)T} \rangle + \frac{\beta}{2} \|\mathbf{A}^{(t)} \mathbf{B}^{(t)T}\|_F^2 \\ &\leq \mathcal{L}(\mathbf{W}^{(t)}) + \langle \nabla \mathcal{L}(\mathbf{W}^{(t)}), \mathbf{A}^{(t)} \mathbf{B}^{(t)T} \rangle + \frac{\beta}{2} \|\mathbf{A}^{(t)}\|_F^2 \|\mathbf{B}^{(t)}\|_F^2 \end{aligned}$$

3) Using the XGBLoRA Gradient Approximation Lemma:

$$\mathbf{A}^{(t)} \mathbf{B}^{(t)T} = \nabla_{\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}} \mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}) + \mathbf{E}^{(t)}$$

where $\|\mathbf{E}^{(t)}\|_F \leq \frac{C_1}{\sqrt{r}} + \frac{C_2}{\sqrt{M}}$.

4) Substituting this into the inequality from step 2:

$$\begin{aligned} \mathcal{L}(\mathbf{W}^{(t+1)}) &\leq \mathcal{L}(\mathbf{W}^{(t)}) + \langle \nabla \mathcal{L}(\mathbf{W}^{(t)}), \nabla_{\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}} \mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}) + \mathbf{E}^{(t)} \rangle \\ &\quad + \frac{\beta}{2} \|\mathbf{A}^{(t)}\|_F^2 \|\mathbf{B}^{(t)}\|_F^2 \end{aligned}$$

5) Using the Gradient Lipschitz Continuity Lemma:

$$\|\nabla \mathcal{L}(\mathbf{W}^{(t)}) - \nabla_{\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}} \mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T})\|_F \leq L' \|\mathbf{A}^{(t)} \mathbf{B}^{(t)T}\|_F$$

6) Applying Cauchy-Schwarz inequality and the bound from step 5:

$$\begin{aligned} \mathcal{L}(\mathbf{W}^{(t+1)}) &\leq \mathcal{L}(\mathbf{W}^{(t)}) - \|\nabla_{\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}} \mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T})\|_F^2 \\ &\quad + L' \|\mathbf{A}^{(t)} \mathbf{B}^{(t)T}\|_F^2 + \|\nabla \mathcal{L}(\mathbf{W}^{(t)})\|_F \|\mathbf{E}^{(t)}\|_F + \frac{\beta}{2} \|\mathbf{A}^{(t)}\|_F^2 \|\mathbf{B}^{(t)}\|_F^2 \end{aligned}$$

7) Using the Accumulated Update Bound Lemma and the gradient bound G:

$$\begin{aligned} \mathcal{L}(\mathbf{W}^{(t+1)}) &\leq \mathcal{L}(\mathbf{W}^{(t)}) - (1 - L\eta_m^2 M^2 G^2 - \frac{\beta}{2} \eta_m^2 M^2 G^2) \|\nabla_{\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}} \mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T})\|_F^2 \\ &\quad + G(\frac{C_1}{\sqrt{r}} + \frac{C_2}{\sqrt{M}}) \end{aligned}$$

8) By μ -strong convexity of $\mathcal{L}$:

$$\|\nabla_{\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}} \mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T})\|_F^2 \geq 2\mu(\mathcal{L}(\mathbf{W}^{(t)} + \mathbf{A}^{(t)} \mathbf{B}^{(t)T}) - \mathcal{L}^*)$$

9) Substituting this into the inequality from step 7:

$$\mathcal{L}(\mathbf{W}^{(t+1)}) - \mathcal{L}^* \leq (1 - 2\mu\alpha)(\mathcal{L}(\mathbf{W}^{(t)}) - \mathcal{L}^*) + G(\frac{C_1}{\sqrt{r}} + \frac{C_2}{\sqrt{M}})$$

where $\alpha = 1 - L\eta_m^2 M^2 G^2 - \frac{\beta}{2} \eta_m^2 M^2 G^2$.

10) Taking expectation and applying this inequality recursively for T iterations:

$$\mathbb{E}[\mathcal{L}(\mathbf{W}^{(T)}) - \mathcal{L}^*] \leq (1 - 2\mu\alpha)^T (\mathcal{L}(\mathbf{W}^{(0)}) - \mathcal{L}^*) + \frac{G}{2\mu\alpha} (\frac{C_1}{\sqrt{r}} + \frac{C_2}{\sqrt{M}})$$

11) Using the inequality $(1-x)^T \leq \exp(-xT) \leq \frac{1}{xT}$ for $x \in (0, 1)$:

$$\mathbb{E}[\mathcal{L}(\mathbf{W}^{(T)}) - \mathcal{L}^*] \leq \frac{C_3}{\sqrt{T}} + \frac{C_4}{M\sqrt{T}} + \frac{C_5}{r}$$

where $C_3 = \frac{(\mathcal{L}(\mathbf{W}^{(0)}) - \mathcal{L}^*)}{2\mu\alpha}$, $C_4 = \frac{GC_2}{2\mu\alpha}$, and $C_5 = \frac{GC_1}{2\mu\alpha}$.

This completes the proof. $\square$

F Detailed Proof of XGBLoRA Expressiveness Theorem

Theorem F.1 (XGBLoRA Expressiveness). *Let f^* be any function in the original function class, and f_T be the function represented by the XGBLoRA-updated network after T iterations. Then:*

$$\mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[(f_T(\mathbf{x}) - f^*(\mathbf{x}))^2] \leq C_6 \left(\frac{1}{r} + \frac{1}{M\sqrt{T}} + \frac{1}{\sqrt{T}} \right)$$

where C_6 is a constant depending on the network architecture, the Lipschitz constants of the activation functions, and L .

PROOF. 1) Let $\mathbf{W}^*$ be the weights that exactly represent f^* in the original function class.

2) Define f_{opt} as the best function that can be represented by XGBLoRA updates:

$$f_{\text{opt}} = \operatorname{argmin}_{f \in \mathcal{F}_{\text{XGBLoRA}}} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[(f(\mathbf{x}) - f^*(\mathbf{x}))^2]$$

where $\mathcal{F}_{\text{XGBLoRA}}$ is the class of functions representable by XGBLoRA updates.

3) We can decompose the error as:

$$\begin{aligned} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[(f_T(\mathbf{x}) - f^*(\mathbf{x}))^2] &\leq 2\mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[(f_T(\mathbf{x}) - f_{\text{opt}}(\mathbf{x}))^2] + 2\mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[(f_{\text{opt}}(\mathbf{x}) - f^*(\mathbf{x}))^2] \\ &= 2E_1 + 2E_2 \end{aligned}$$

4) For E_1 , we can use the Convergence Theorem (Theorem 1):

$$E_1 \leq K_1 \left(\frac{1}{\sqrt{T}} + \frac{1}{M\sqrt{T}} \right)$$

where K_1 is a constant related to C_3 and C_4 from Theorem 1.

5) For E_2 , we need to analyze how well XGBLoRA updates can approximate $\mathbf{W}^*$. Let $\Delta\mathbf{W} = \mathbf{W}^* - \mathbf{W}^{(0)}$.

6) We can approximate $\Delta\mathbf{W}$ with a sequence of low-rank updates:

$$\Delta\mathbf{W} \approx \sum_{t=1}^T \mathbf{A}^{(t)} (\mathbf{B}^{(t)})^T$$

7) By the properties of low-rank matrix approximation:

$$\|\Delta\mathbf{W} - \sum_{t=1}^T \mathbf{A}^{(t)} (\mathbf{B}^{(t)})^T\|_F \leq \frac{\|\Delta\mathbf{W}\|_*}{\sqrt{rT}}$$

where $\|\cdot\|_*$ denotes the nuclear norm.

8) Assuming the network function is Lipschitz continuous with respect to its weights with Lipschitz constant L_f :

$$E_2 \leq L_f^2 \|\Delta\mathbf{W} - \sum_{t=1}^T \mathbf{A}^{(t)} (\mathbf{B}^{(t)})^T\|_F^2 \leq \frac{L_f^2 \|\Delta\mathbf{W}\|_*^2}{rT}$$

9) Combining the bounds for E_1 and E_2 :

$$\begin{aligned} \mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[(f_T(\mathbf{x}) - f^*(\mathbf{x}))^2] &\leq 2K_1 \left(\frac{1}{\sqrt{T}} + \frac{1}{M\sqrt{T}} \right) + \frac{2L_f^2 \|\Delta\mathbf{W}\|_*^2}{rT} \\ &\leq C_6 \left(\frac{1}{r} + \frac{1}{M\sqrt{T}} + \frac{1}{\sqrt{T}} \right) \end{aligned}$$

where $C_6 = \max(2K_1, 2L_f^2 \|\Delta\mathbf{W}\|_*^2)$.

This completes the proof. $\square$

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009